

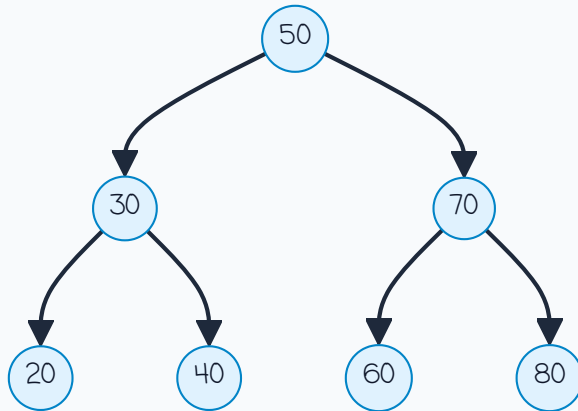
Data Structures & Algorithms

Binary Search Trees (BST) & Traversals

BINARY SEARCH TREE (BST) & TREE TRAVERSALS

A **Binary Search Tree** is a node-based binary tree data structure where each node has at most two children.

Visual Binary Search Tree (BST)



Key Properties of BST

- **Left Subtree:** Values are strictly smaller than root (**Left < Root**).
- **Right Subtree:** Values are strictly greater than root (**Right > Root**).
- **Inorder Traversal:** Produces elements in **sorted ascending order**.

Time Complexity Analysis

Operation	Average Case	Worst Case
Search	$O(\log N)$	$O(N)$
Insertion	$O(\log N)$	$O(N)$
Deletion	$O(\log N)$	$O(N)$

Space for Notes

Data Structures & Algorithms

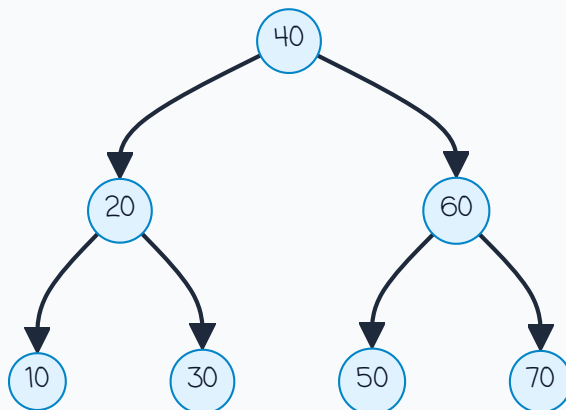
Binary Search Trees (BST) & Traversals

C++ IMPLEMENTATION OF BST INSERTION

CPP

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
    Node(int val) : data(val), left(nullptr), right(nullptr) {}  
};  
  
Node* insertBST(Node* root, int val) {  
    if (root == nullptr) return new Node(val);  
    if (val < root->data) root->left = insertBST(root->left, val);  
    else root->right = insertBST(root->right, val);  
    return root;  
}
```

Balanced AVL Tree



Space for Notes

